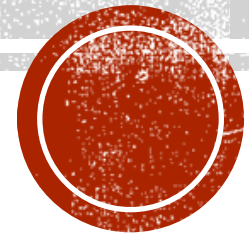


DOCKERIZE THE HELLO-SERVER APPLICATION

Done by : Shahd AL Abdali

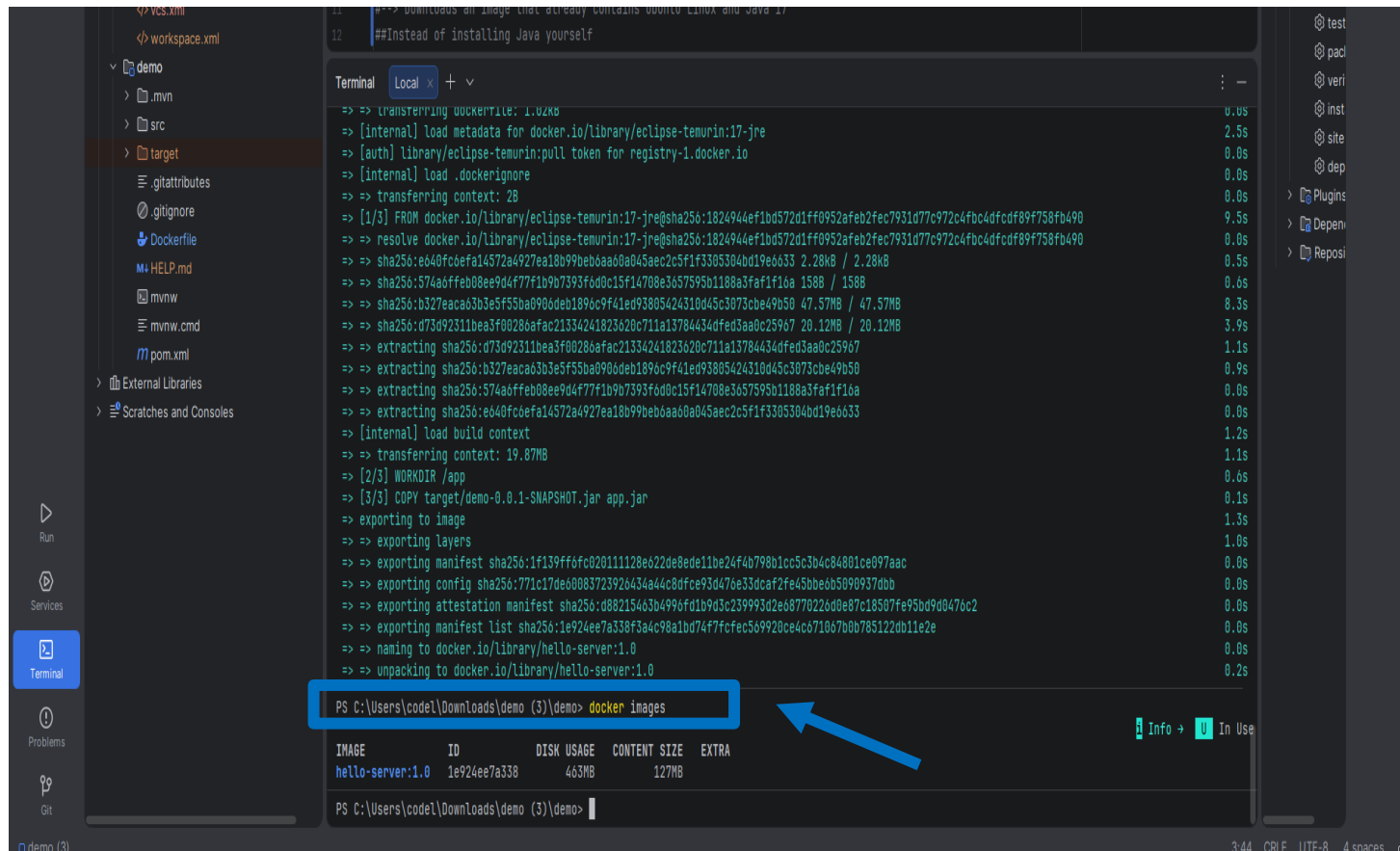


DOCKERIZE THE APPLICATION

- Build Docker image
- Using the same file that used in VM(last task)
- Run container
- Verify application
- Create Docker Compose
- Test Compose lifecycle



DOCKERFILE - BUILD DOCKER IMAGE



The screenshot shows an IDE with a Dockerfile in the editor and a terminal window displaying the build process. The Dockerfile contains the following instructions:

```
FROM ubuntu:18.04
MAINTAINER jre@sha256:1824944ef1bd572d1ff0952afab2fec7931d77c972c4fbc4dfcd89f758fb490
#Instead of installing Java yourself
```

The terminal output shows the build steps and progress:

```
=> transferring dockerfile: 1.02kB
=> [internal] load metadata for docker.io/library/eclipse-temurin:17-jre
=> [auth] library/eclipse-temurin:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> transferring context: 28
=> [1/3] FROM docker.io/library/eclipse-temurin:17-jre@sha256:1824944ef1bd572d1ff0952afab2fec7931d77c972c4fbc4dfcd89f758fb490
=> resolve docker.io/library/eclipse-temurin:17-jre@sha256:1824944ef1bd572d1ff0952afab2fec7931d77c972c4fbc4dfcd89f758fb490
=> sha256:eb40fc6efa14572a4927ea18b99bebbaa0a045aec2c5f1f3305304bd19e6633 2.28kB / 2.28kB
=> sha256:574a6ffeb08ee9d4f77f1b9b7393f0d0c15f14708e3657595b1188a3faf1f16a 158B / 158B
=> sha256:b327eaca63b3e5f55ba90b0deb1896c9f41ed93805424310d45c3073cbe49b50 47.57MB / 47.57MB
=> sha256:d73d92311bea3f00280afac21334241823620c711a13784434dfed3aa0c25967 20.12MB / 20.12MB
=> extracting sha256:d73d92311bea3f00280afac21334241823620c711a13784434dfed3aa0c25967
=> extracting sha256:b327eaca63b3e5f55ba90b0deb1896c9f41ed93805424310d45c3073cbe49b50
=> extracting sha256:574a6ffeb08ee9d4f77f1b9b7393f0d0c15f14708e3657595b1188a3faf1f16a
=> extracting sha256:eb40fc6efa14572a4927ea18b99bebbaa0a045aec2c5f1f3305304bd19e6633
=> [internal] load build context
=> transferring context: 19.87MB
=> [2/3] WORKDIR /app
=> [3/3] COPY target/demo-0.0.1-SNAPSHOT.jar app.jar
=> exporting to image
=> exporting layers
=> exporting manifest sha256:1f139ff6c02011128e622de8ede11be24f4b798b1cc5c3b4c84801ce097aac
=> exporting config sha256:771c17de6008372326434a4c8dfce93d476e33dcacf2fe45bbe6b50909370bb
=> exporting attestation manifest sha256:088215463b4996fd1b9d3c239993d2e08770220d0e87c18507fe95bd9d0476c2
=> exporting manifest list sha256:1e924ee7a338f3a4c98a1bd747f7cfec569920ce4c671067b0b785122db11e2e
=> naming to docker.io/library/hello-server:1.0
=> unpacking to docker.io/library/hello-server:1.0
```

The terminal also shows the command to list Docker images:

```
PS C:\Users\code1\Downloads\demo (3)\demo> docker images
```

The output of the command is shown in a table:

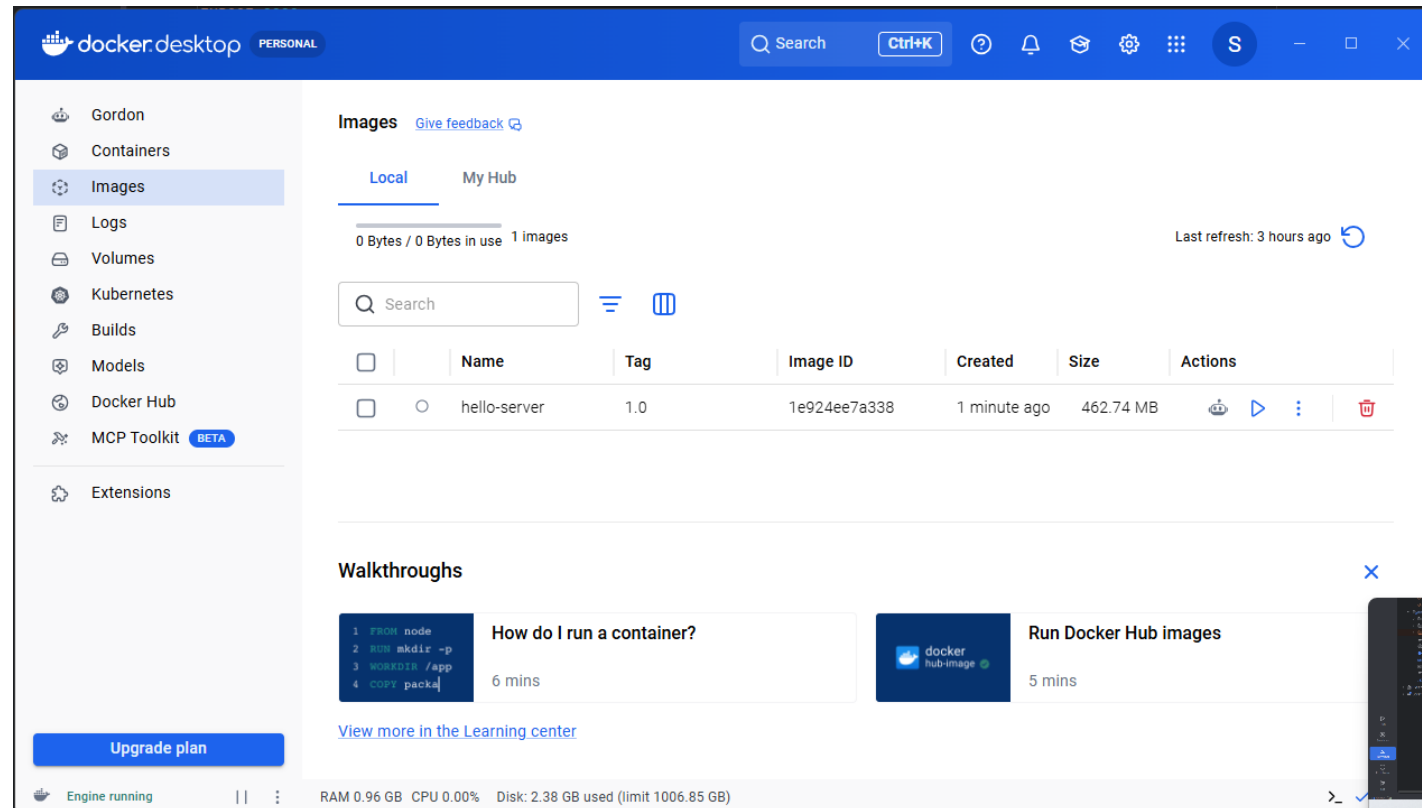
IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
hello-server:1.0	1e924ee7a338	463MB	127MB	

docker build -t hello-server:1.0 .

Verify Image.



DOCKERFILE - BUILD DOCKER IMAGE



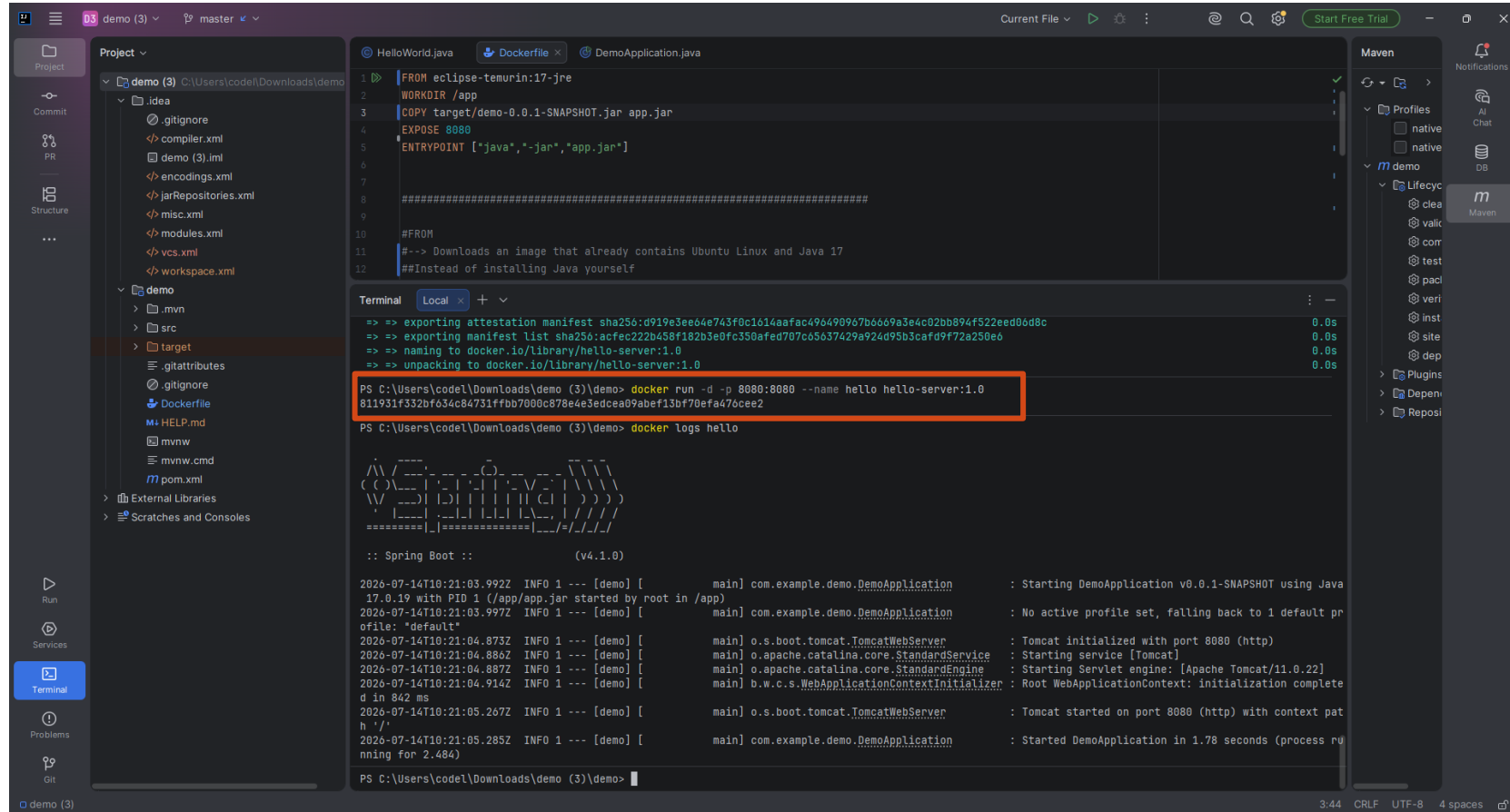
The screenshot shows the Docker Desktop application window. The left sidebar contains navigation links: Gordon, Containers, Images (selected), Logs, Volumes, Kubernetes, Builds, Models, Docker Hub, MCP Toolkit (BETA), and Extensions. The main area is titled 'Images' and has tabs for 'Local' and 'My Hub'. Below the tabs, it shows '0 Bytes / 0 Bytes in use' and '1 Images'. A search bar and view toggles are present. A table lists the local images:

	Name	Tag	Image ID	Created	Size	Actions
☐	hello-server	1.0	1e924ee7a338	1 minute ago	462.74 MB	

Below the table is a 'Walkthroughs' section with two cards: 'How do I run a container?' (6 mins) and 'Run Docker Hub images' (5 mins). A 'View more in the Learning center' link is at the bottom. The status bar at the bottom shows 'Engine running', RAM usage (0.96 GB), CPU usage (0.00%), and Disk usage (2.38 GB used / 1006.85 GB limit).



RUN CONTAINER



The screenshot shows an IDE interface with a project named 'demo'. The 'Project' view on the left shows the file structure, including 'demo (3)' and 'demo'. The 'Maven' view on the right shows the Maven lifecycle. The 'Terminal' view at the bottom shows the execution of Docker commands and application logs.

```
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY target/demo-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]

#FROM
#--> Downloads an image that already contains Ubuntu Linux and Java 17
##Instead of installing Java yourself
```

```
PS C:\Users\code\Downloads\demo (3)\demo> docker run -d -p 8080:8080 --name hello hello-server:1.0
811931f332bf634c84731ffbb7000c878e4e3edcea09abef13bf70efa476cee2
```

```
PS C:\Users\code\Downloads\demo (3)\demo> docker logs hello
```

```

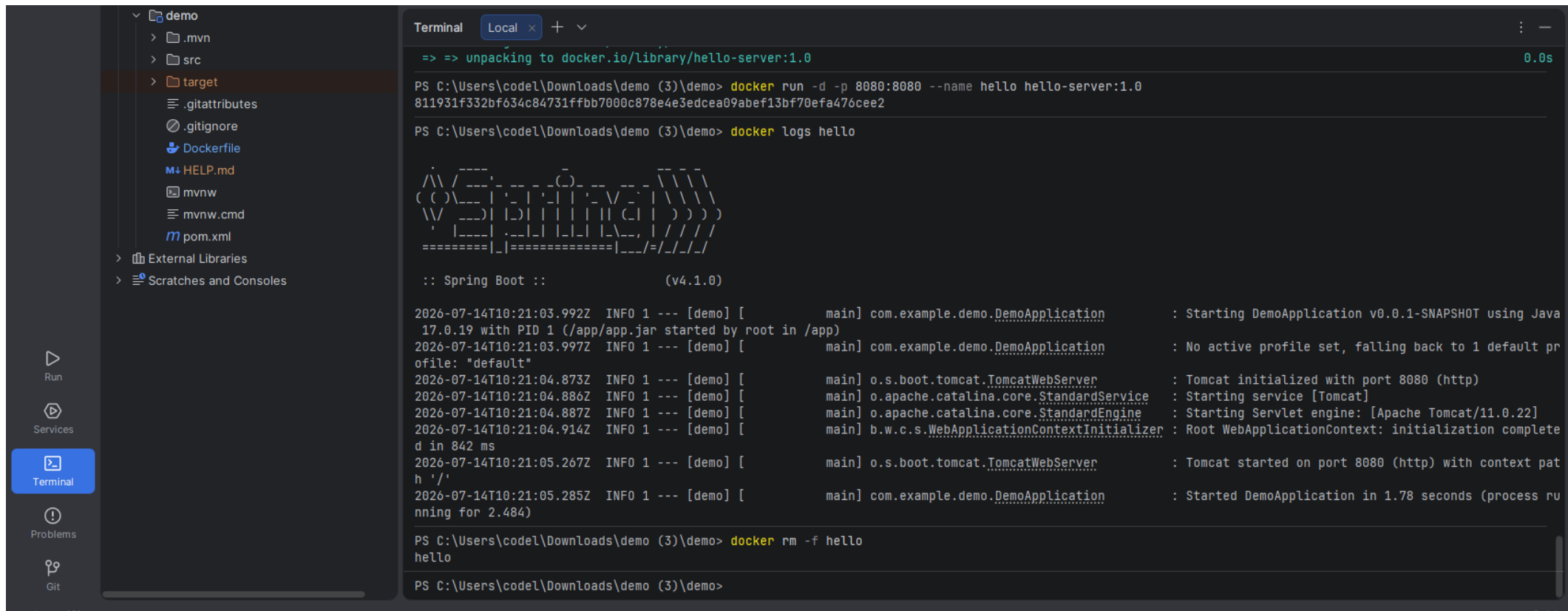
  ____  __
 / ___/ /_  __
/ /   / __/ /
/ /___/ /_ /
/____/_/  /

:: Spring Boot ::                (v4.1.0)

2026-07-14T10:21:03.992Z INFO 1 --- [demo] [main] com.example.demo.DemoApplication : Starting DemoApplication v0.0.1-SNAPSHOT using Java
17.0.19 with PID 1 (/app/app.jar started by root in /app)
2026-07-14T10:21:03.997Z INFO 1 --- [demo] [main] com.example.demo.DemoApplication : No active profile set, falling back to 1 default pr
ofiles: "default"
2026-07-14T10:21:04.873Z INFO 1 --- [demo] [main] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-07-14T10:21:04.886Z INFO 1 --- [demo] [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-07-14T10:21:04.887Z INFO 1 --- [demo] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.22]
2026-07-14T10:21:04.914Z INFO 1 --- [demo] [main] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization complete
d in 842 ms
2026-07-14T10:21:05.267Z INFO 1 --- [demo] [main] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context pat
h '/'
2026-07-14T10:21:05.285Z INFO 1 --- [demo] [main] com.example.demo.DemoApplication : Started DemoApplication in 1.78 seconds (process ru
nning for 2.484s)
```



VERIFY APPLICATION LOGS



The screenshot shows an IDE interface with a file explorer on the left and a terminal window on the right. The file explorer shows a project named 'demo' with a 'target' directory selected. The terminal window displays the following content:

```
Terminal Local x + v
=> => unpacking to docker.io/library/hello-server:1.0 0.0s

PS C:\Users\code1\Downloads\demo (3)\demo> docker run -d -p 8080:8080 --name hello hello-server:1.0
811931f332bf634c84731ffbb7000c878e4e3edcea09abef13bf70efa476cee2

PS C:\Users\code1\Downloads\demo (3)\demo> docker logs hello

  ____ _
 / ___ \| | | |
/ /   \| |_| |
\ \   /| | | |
 \___/\|_|_|_|

:: Spring Boot ::                (v4.1.0)

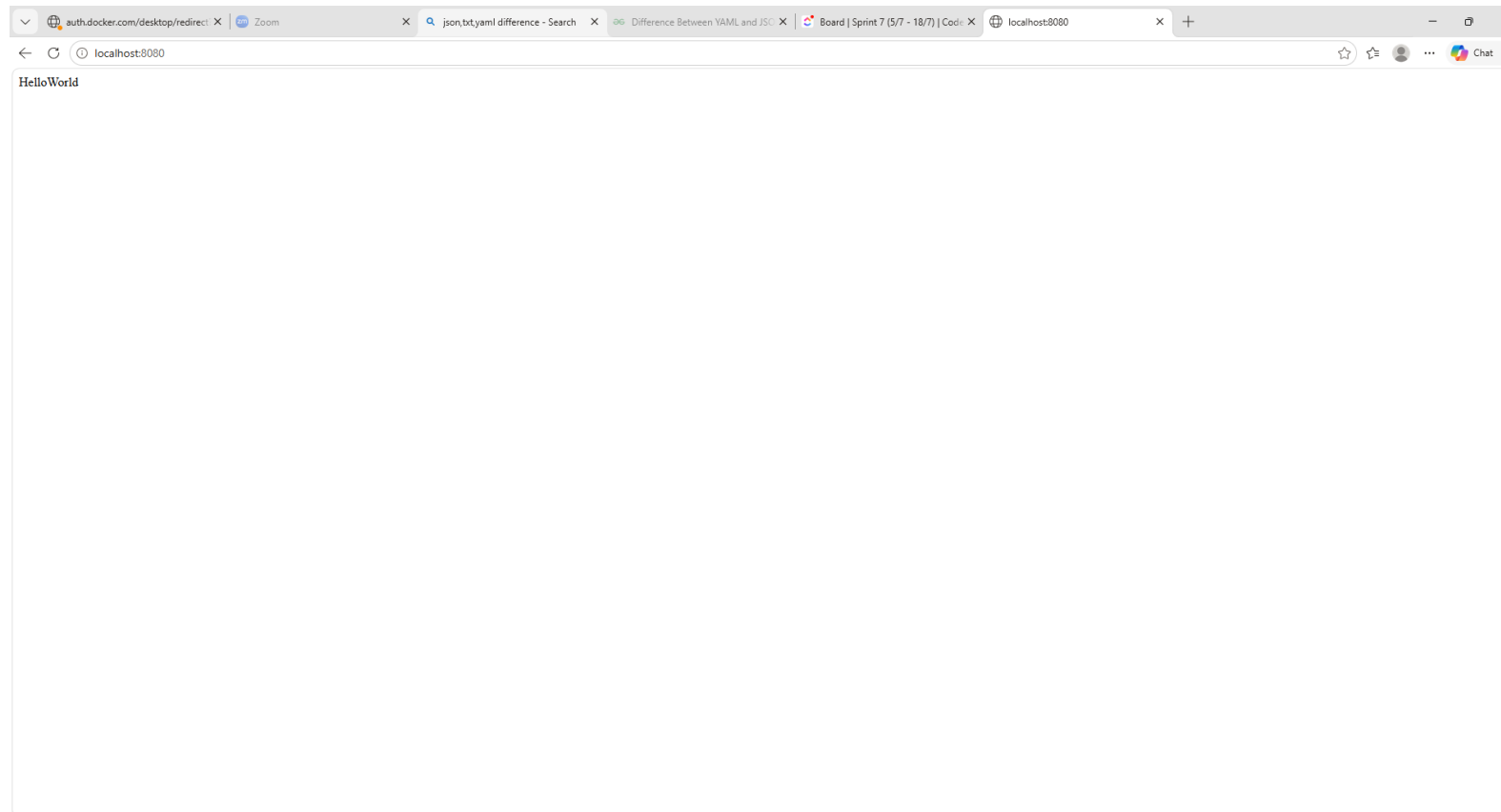
2026-07-14T10:21:03.992Z INFO 1 --- [demo] [main] com.example.demo.DemoApplication : Starting DemoApplication v0.0.1-SNAPSHOT using Java
17.0.19 with PID 1 (/app/app.jar started by root in /app)
2026-07-14T10:21:03.997Z INFO 1 --- [demo] [main] com.example.demo.DemoApplication : No active profile set, falling back to 1 default pr
ofile: "default"
2026-07-14T10:21:04.873Z INFO 1 --- [demo] [main] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-07-14T10:21:04.886Z INFO 1 --- [demo] [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-07-14T10:21:04.887Z INFO 1 --- [demo] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.22]
2026-07-14T10:21:04.914Z INFO 1 --- [demo] [main] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization complete
d in 842 ms
2026-07-14T10:21:05.267Z INFO 1 --- [demo] [main] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context pat
h '/'
2026-07-14T10:21:05.285Z INFO 1 --- [demo] [main] com.example.demo.DemoApplication : Started DemoApplication in 1.78 seconds (process ru
nning for 2.484)

PS C:\Users\code1\Downloads\demo (3)\demo> docker rm -f hello
hello

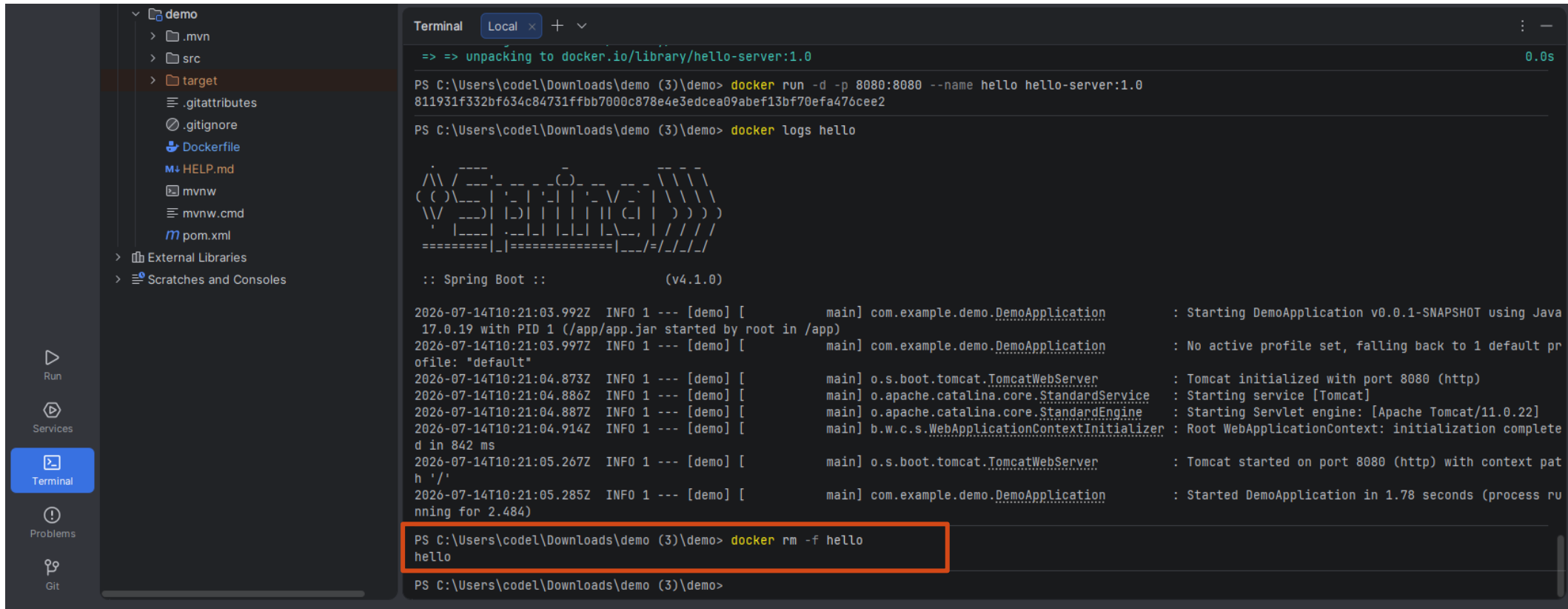
PS C:\Users\code1\Downloads\demo (3)\demo>
```



APPLICATION RUNNING



REMOVE CONTAINER



The screenshot shows an IDE interface with a file explorer on the left and a terminal window on the right. The file explorer shows a project named 'demo' with a 'target' folder selected. The terminal window shows the following commands and output:

```
=> => unpacking to docker.io/library/hello-server:1.0 0.0s

PS C:\Users\codel\Downloads\demo (3)\demo> docker run -d -p 8080:8080 --name hello hello-server:1.0
811931f332bf634c84731ffbb7000c878e4e3edcea09abef13bf70efa476cee2

PS C:\Users\codel\Downloads\demo (3)\demo> docker logs hello

  ____  _
 / ___|| | | |
| |___| |_| |
 \___ \|  _/
      |_| |_|

:: Spring Boot ::                (v4.1.0)

2026-07-14T10:21:03.992Z INFO 1 --- [demo] [main] com.example.demo.DemoApplication : Starting DemoApplication v0.0.1-SNAPSHOT using Java
17.0.19 with PID 1 (/app/app.jar started by root in /app)
2026-07-14T10:21:03.997Z INFO 1 --- [demo] [main] com.example.demo.DemoApplication : No active profile set, falling back to 1 default pr
ofile: "default"
2026-07-14T10:21:04.873Z INFO 1 --- [demo] [main] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-07-14T10:21:04.886Z INFO 1 --- [demo] [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-07-14T10:21:04.887Z INFO 1 --- [demo] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.22]
2026-07-14T10:21:04.914Z INFO 1 --- [demo] [main] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization complete
d in 842 ms
2026-07-14T10:21:05.267Z INFO 1 --- [demo] [main] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context pat
h '/'
2026-07-14T10:21:05.285Z INFO 1 --- [demo] [main] com.example.demo.DemoApplication : Started DemoApplication in 1.78 seconds (process ru
nning for 2.484)

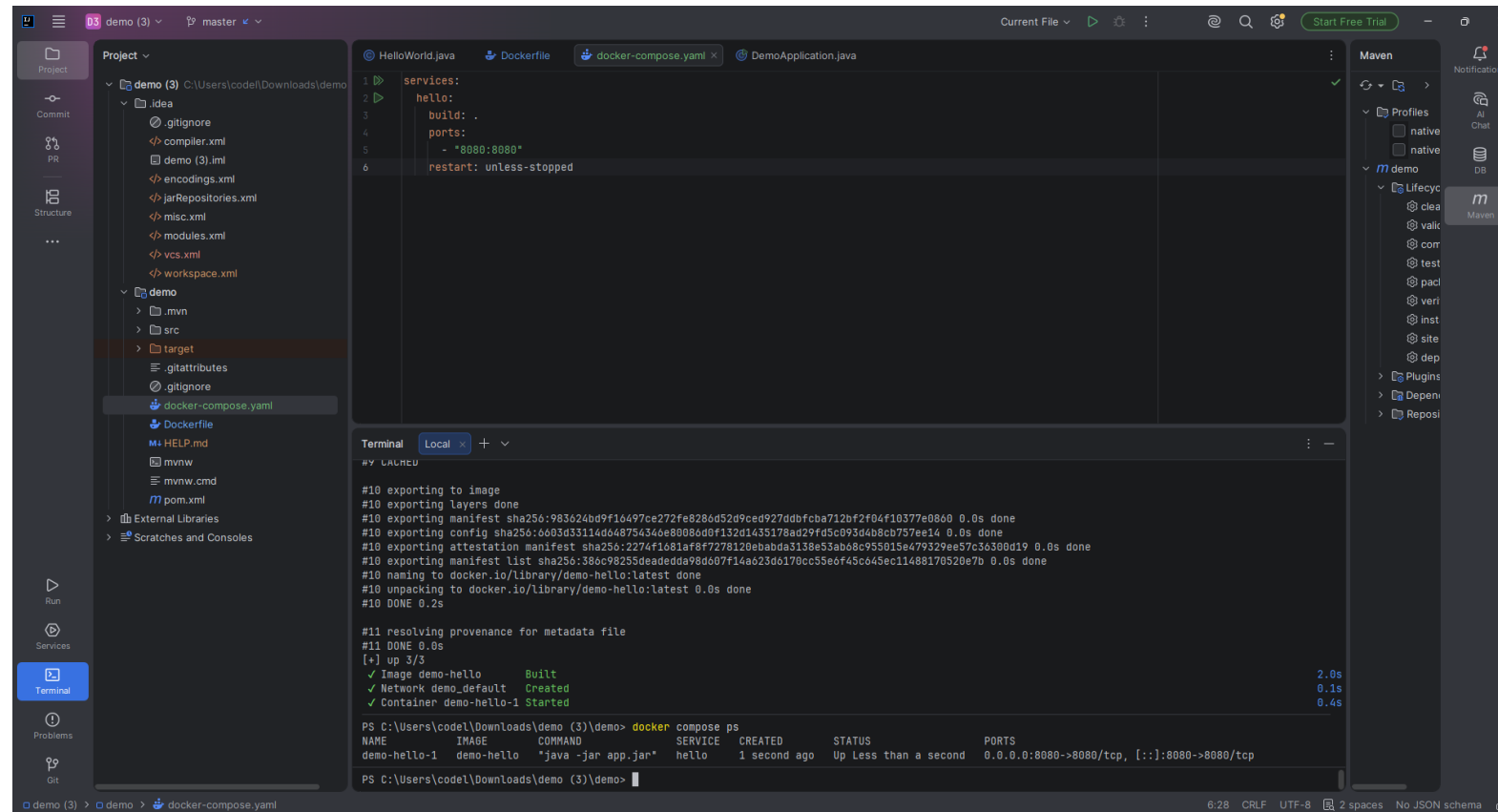
PS C:\Users\codel\Downloads\demo (3)\demo> docker rm -f hello
hello

PS C:\Users\codel\Downloads\demo (3)\demo>
```

The command `docker rm -f hello` is highlighted with a red box, indicating the removal of the container.



DOCKER COMPOSE CONFIGURATION + DOCKER COMPOSE UP



The screenshot shows an IDE interface with the following components:

- Project Explorer:** Shows a project named 'demo (3)' with files like `gitignore`, `compiler.xml`, `demo (3).iml`, `encodings.xml`, `jarRepositories.xml`, `misc.xml`, `modules.xml`, `vcs.xml`, `workspace.xml`, `demo`, `mvn`, `src`, `target`, `gitattributes`, `gitignore`, `docker-compose.yaml`, `Dockerfile`, `HELP.md`, `mvnw`, `mvnw.cmd`, `pom.xml`, `External Libraries`, and `Scratches and Consoles`.
- Editor:** Displays the `docker-compose.yaml` file with the following content:

```
services:
  hello:
    build: .
    ports:
      - "8080:8080"
    restart: unless-stopped
```
- Terminal:** Shows the output of the `docker compose up` command. The output includes progress bars for exporting layers, manifest, config, and attestation, and a table showing the status of the containers.

```
#10 exporting to image
#10 exporting layers done
#10 exporting manifest sha256:983624bd9f16497ce272fe8286d52d9ced927ddbfcba712bf2f04f10377e8860 0.0s done
#10 exporting config sha256:6603d33114d048754346e80886d9f132d1435178ad29fd5c093d4b8cb757ee14 0.0s done
#10 exporting attestation manifest sha256:2274f1081af8f7278120ebabba3138e53ab08c955015e479329ee57c36308d19 0.0s done
#10 exporting manifest list sha256:380c98255deadedda98d607f14a623d0170cc55e6f45c64Sec11488170520e7b 0.0s done
#10 naming to docker.io/library/demo-hello:latest done
#10 unpacking to docker.io/library/demo-hello:latest 0.0s done
#10 DONE 0.2s

#11 resolving provenance for metadata file
#11 DONE 0.0s
[+] up 3/3
✓ Image demo-hello Built 2.0s
✓ Network demo_default Created 0.1s
✓ Container demo-hello-1 Started 0.4s

PS C:\Users\code\Downloads\demo (3)\demo> docker compose ps
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
demo-hello-1 demo-hello "java -jar app.jar" hello 1 second ago Up Less than a second 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp

PS C:\Users\code\Downloads\demo (3)\demo>
```
- Maven:** Shows the Maven lifecycle with various goals like `clean`, `validate`, `compile`, `test`, `package`, `verify`, `install`, `site`, `deploy`, `Plugins`, `Dependencies`, and `Repositories`.



RESTART COMPOSE STACK

The screenshot shows an IDE interface with the following components:

- Project Explorer:** Shows a project named 'demo (3)' with files like `docker-compose.yml`, `Dockerfile`, and `demo (3).iml`.
- Editor:** Displays the `docker-compose.yml` file with the following content:

```
services:
  hello:
    build: .
    ports:
      - "8080:8080"
    restart: unless-stopped
```
- Terminal:** Shows the execution of Docker Compose commands:

```
PS C:\Users\code\Downloads\demo (3)\demo> docker compose down
[+] down 2/2
✓ Container demo-hello-1 Removed 0.5s
✓ Network demo_default Removed 0.2s

PS C:\Users\code\Downloads\demo (3)\demo> docker compose up -d
[+] up 2/2
✓ Network demo_default Created 0.0s
✓ Container demo-hello-1 Started 0.3s

PS C:\Users\code\Downloads\demo (3)\demo> docker compose ps
NAME          IMAGE          SERVICE   CREATED      STATUS      PORTS
demo-hello-1  demo-hello     hello     1 second ago Up Less than a second 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
```
- Right Panel:** Shows the Maven tool window with a list of profiles and dependencies.



RESULT

- Successfully containerized the Spring Boot application using Docker and Docker Compose. The application was accessible on port 8080 and verified through browser testing.

